Where to Classify?

Exploring Classification Across the Legal RAG Pipeline

Problem and objective

Legal retrieval-augmented generation (RAG) requires statutory provisions that address a user's situation. Laypeople describe everyday problems, whereas statutes express legal concepts, conditions, and exceptions. This language gap can make relevant provisions difficult to retrieve, leaving generated answers insufficiently grounded. Improving retrieval could lower the barrier to accessing legal information. We ask: **How do classification-based interventions at different stages of hybrid retrieval affect retrieval quality and computational cost?** We study article-level retrieval for public legal consultation.

Data and baseline

We use STARD [1], containing 1,543 real-world Chinese consultation queries and 55,348 candidate statutory articles, with multiple relevant articles possible per query. Our shared baseline combines BM25 and a Chinese dense retriever using reciprocal rank fusion (RRF). BM25-only, dense-only, and hybrid retrieval with a pretrained Chinese cross-encoder reranker provide additional controls.

Proposed methods

We investigate three parallel strategies, each independently added to the hybrid baseline:

Before retrieval - multi-law routing. Given a query and descriptions of legal documents, Jev predicts which documents are relevant through separate relevance judgments. Hybrid retrieval then searches their combined article collections. Allowing multiple documents accommodates questions spanning different laws. We measure how many annotated relevant articles survive routing to distinguish routing errors from subsequent retrieval failures.

During indexing and retrieval - shared-concept tagging. GPT constructs a fixed legal concept vocabulary from the corpus and produces multi-label annotations for articles and training queries; a sample is manually checked. A Chinese BERT classifier is fine-tuned on these annotations to assign shared concepts to queries and articles. Tag overlap supplies an additional retrieval signal. Its candidates are merged with BM25 and dense candidates through weighted RRF, allowing concepts to affect candidate coverage as well as ranking. Fusion weights are tuned on validation data.

After retrieval - Jev shortlist selection. Jev receives the query and the hybrid top-n articles and makes an n-way choice of the most relevant article [2]. We evaluate its selected article and rank the shortlist by its returned option probabilities. This tests whether single-choice preference scores also support retrieval of multiple relevant articles. A conventional reranker uses the same shortlist; candidate coverage determines the ceiling for either approach.

Evaluation

We reserve the repository's development split for final evaluation and divide its training queries into training and validation sets. All strategies share the corpus, base retrievers, and final output size; prompts, thresholds, and fusion weights are selected on validation data. We report Recall@10, nDCG@10, and Hit@1, alongside shortlist Recall@n and routing retention. We vary routing breadth, tag contributions, and shortlist size, and analyze single- versus multiple-relevant-article queries. Efficiency measures include per-query latency, API tokens and fees, and local peak memory. Offline annotation, training, and indexing costs are reported separately. Model versions, prompts, splits, and code will be documented for reproducibility.

Expected contribution

The project will characterize when classification improves legal retrieval and whether its gains justify its costs. Error analysis will connect outcomes to misrouting, missing or incorrect tags, and candidate-selection errors. The comparison concerns complete strategies, which differ in classification task as well as placement; no strategy is assumed to outperform the baseline.

References

[1] Su, W., et al. (2024). STARD: A Chinese Statute Retrieval Dataset Derived from Real-life Queries by Non-professionals. Findings of EMNLP. https://aclanthology.org/2024.findings-emnlp.625/

[2] TypeSafe. Choice: Jev structured selection and option probabilities. https://docs.typesafe.ai/primitives/choice